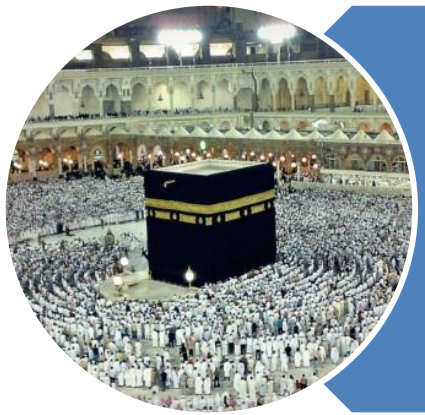
Data Structures - I

CPCS 204

رَبِّ اشْرَحْ لِي صَدْرِي وَيَسِّرْ لِي أَمْرِي وَاحْلُلْ عُقْدَةً مِنْ لِسَانِي يَفْقَهُوا قَوْلِي



O my Lord! Expand for me my chest [with assurance] and ease for me my task and untie the knot from my tongue that they may understand my speech. **(Quran 20 : 25-28)**

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

CPCS 204

Binary Heap

Applications

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

Binary Heap - **Application**

- Priority Queue
- Heap Sort

CPCS 204

Binary Heap Applications

Priority Queues

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

Priority Queues - **Introduction**

- A priority queue is basically what it sounds like
  - it is a queue
  - Which means that we will have a line
  - But the first person in line is not necessarily the first person out of line
  - Rather, the **queuing order is based on a priority**
  - Meaning, if one person has a higher priority, that person goes right to the front
- Examples:
  - Emergency room:
    - Higher priority injuries are taken first

Priority Queues - **Implementation**

- Sorted Linked List / Arrays
  - Higher priority items are ALWAYS at the front of the list
  - Example: a check out line in a supermarket
    - But people who are more important can cut in line
  - Running Time:
    - O(n) insertion time: you have to search through, potentially, n nodes to find the correct spot (based on priority)
    - O(1) deletion time: (finding the node with the highest priority) since the highest priority node is first node of the list

Priority Queues - **Implementation**

- Unsorted Linked List / Arrays
  - To add an element, append it to the end
  - To remove an element, search through all the elements for the one with the highest priority
  - Running Time:
    - O(1) insertion time: you simple add to the end of the list
    - O(n) deletion time: you have to, potentially, search through all n nodes to find the correct node to delete

Priority Queues - **Implementation**

- Binary Search Tree
  - Insertion
    - O (log n)
  - Removal
    - O (log n)
  - Exception
    - If data comes in sorted order
      - O(n) insertion time: you simple add to the end of the list
      - O(n) deletion time: you have to, potentially, search through all n nodes to find the correct node to delete

Priority Queues - **Implementation**

- Binary Heap
  - It ensures to have Complete Binary Tree
  - Running time ends up being **O(log n)** for both insertion and deletion into a Heap
  - It is an ideal choice for implementing priority queues

CPCS 204

Binary Heap Applications

Heap Sort

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

Binary Heap – **Sorting**

- Build max /min heap from array
  - O(n log n)
- Delete value from heap i.e. "root"
  - Minimum or Maximum
  - O(log n)
- It is really just a series of "deletion"
  - Delete all elements one by one from heap
- Store these removed items, sequentially, in an array
- Overall : **O (n log n)**

Binary Heap

Implementation

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

CPCS 204

Implementation - **Introduction**

- Binary Heap is ADT
- Logical Data Structures
- Implementation
  - Linked List
  - Arrays

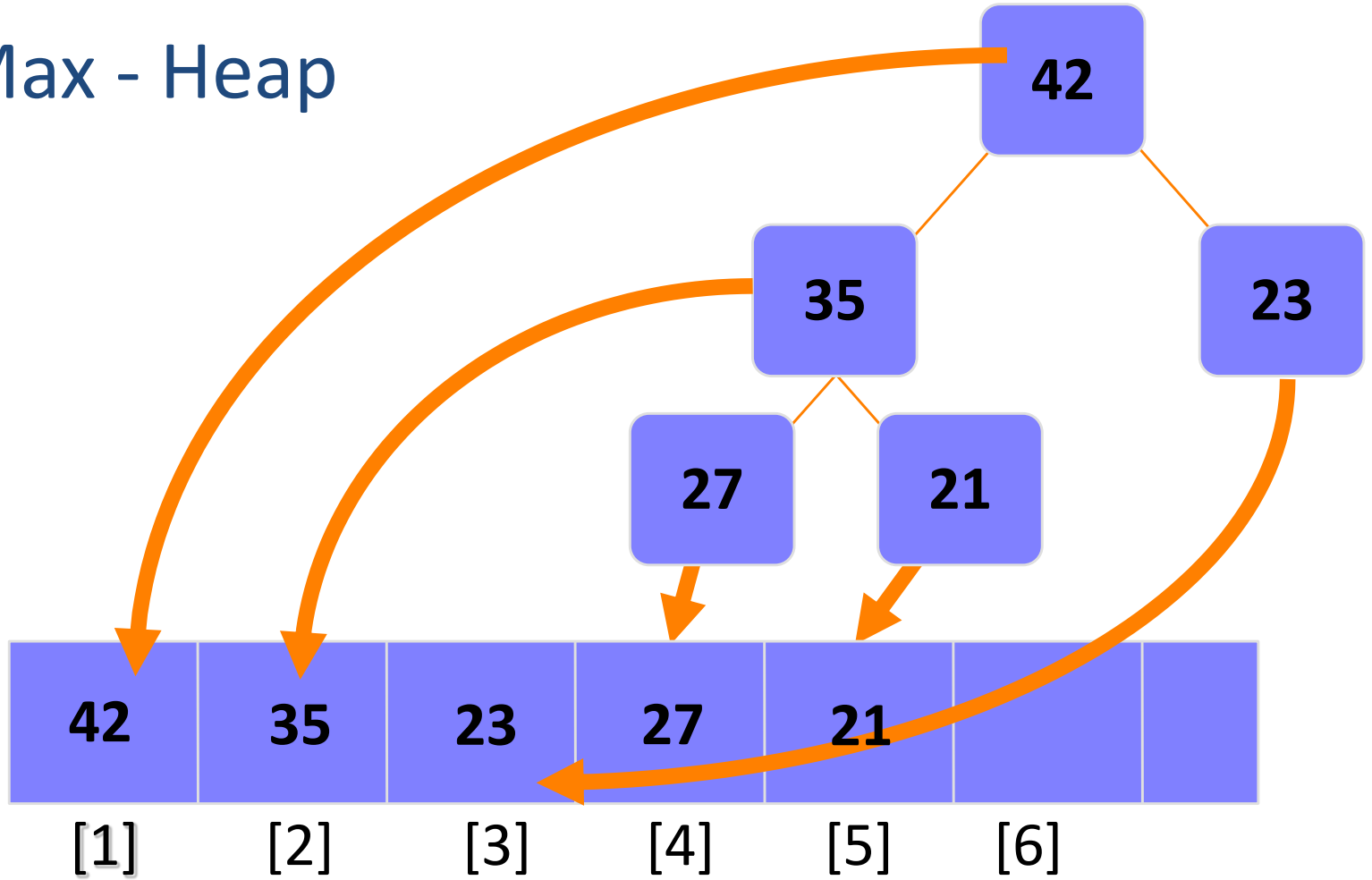
Implementation - **Arrays**

- It is Complete Binary Tree
  - **The root** would be the 1st position of the array (index 1)
  - The **two children** of the node would be in index 2 and 3
  - The **4 nodes on the next level** would be in index 4 – 7
  - The **8 nodes on the next level** would be in index 8 - 15
  - and so on

CPCS 204

Implementation - **Arrays**

- Max - Heap



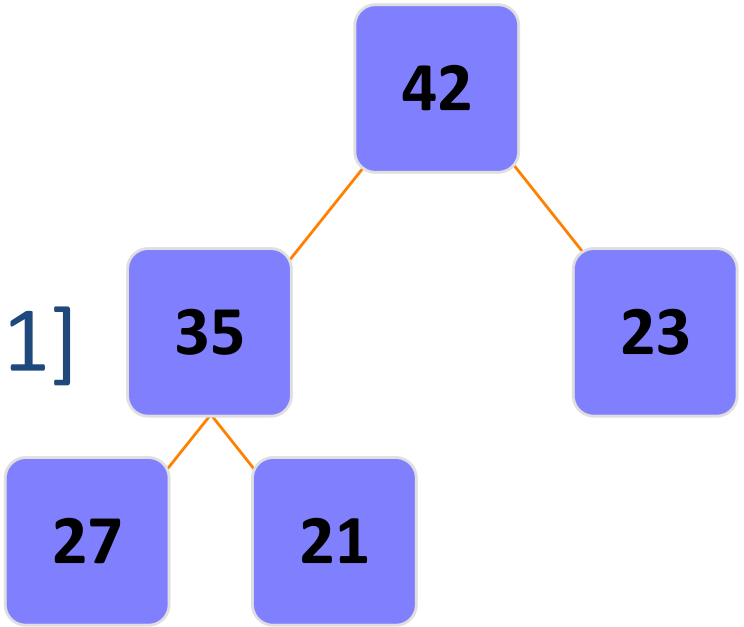
Dr. Jonathan (Yahya) Cazalas

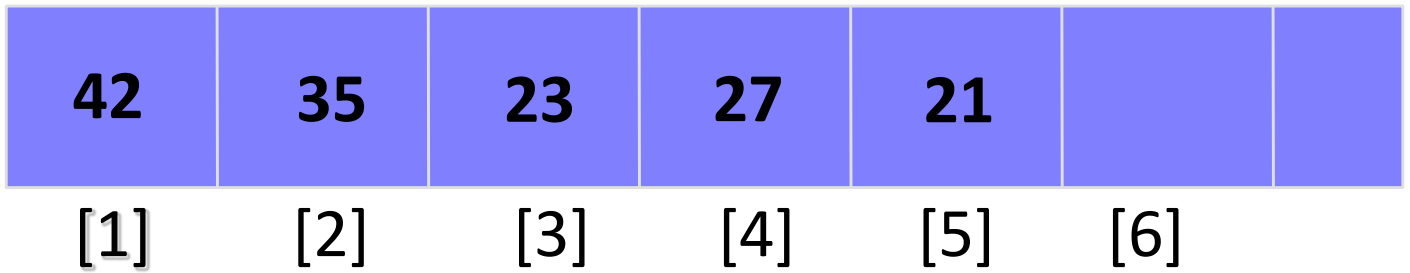
Dr. Muhammad Umair Ramzan

CPCS 204

Implementation - Arrays

- Root is at position A[1]
- Left child of A[i] = A[2i]
- Right child of A[i] = A[2i+1]
- Parent of A[i] = A[i/2]





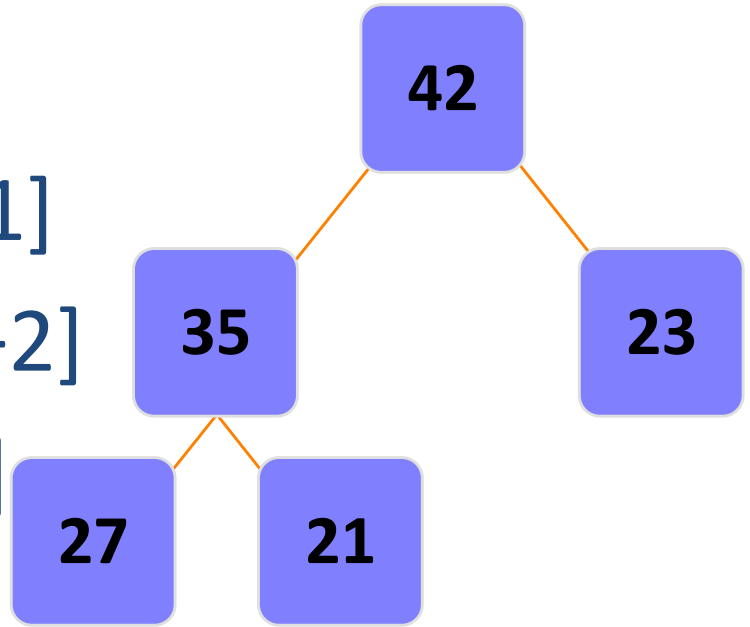
Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

CPCS 204

Implementation - Arrays

- Root is at position A[0]
- Left child of A[i] = A[2i +1]
- Right child of A[i] = A[2i+2]
- Parent of A[i] = A[(i-1)/2]



42	35	23	27	21		
[0]	[1]	[2]	[3]	[4]	[5]	

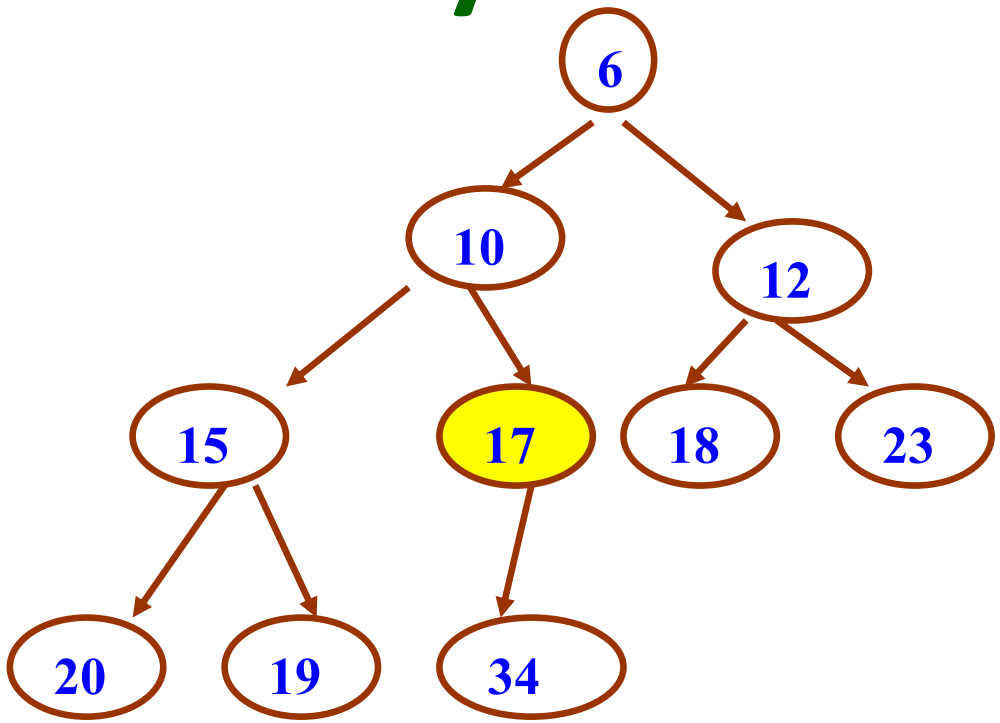
Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

CPCS 204

Implementation - **Arrays**

- Consider node **17**:
  - Position : 5
  - Parent is 10
    - Position [5/2] = 2
  - Left child 34
    - Position 5*2 = 10
  - Right child - **None**
    - Position
    - 2*5 + 1 = 11
    - empty



6	10	12	15	17	18	23	20	19	34	
[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

Binary Heap - **Summary**

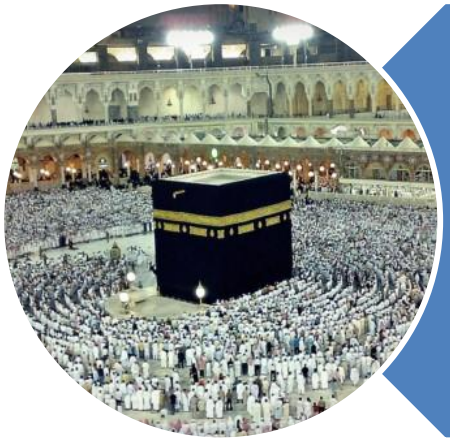
- A binary heap is a tree that satisfies 2 properties:
  - The Heap Property
    - Max-heap OR Min-heap
  - The Shape Property
    - Must be a complete binary tree
- To add elements to a heap
  - Place element at next available spot and Percolate Up
- To remove elements from a heap,
  - Delete root, as it is always the one you want to remove
  - Then copy last element to root's position
  - Finally, Percolate Down
- Applications
  - Priority Queues
  - Heap Sort

Data Structures - I

"No Bonus" marks for this course anymore

CPCS 204

20



Allah (Alone) is Sufficient for us, and He is the Best Disposer of affairs (for us).(Quran 3 : 173)

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan